Software Requirements Specification (SRS)

Healthcare Management System with Live Token & Appointment Scheduling

Prepared by:

- Krish Gupta (Roll No: 24293916007)
- Yuvraj Dahiya (Roll No: 24293916069)
- Amandeep Singh (Roll No: 24293916067)
- Mukul Chauhan (Roll No: 24293916062)
- Daksh Dahiya (Roll No: 24293916060)
- Himanshu Chhillar (Roll No: 24293916028)

Organisation: Faculty of Technology

Team Name: Wait Zero

Date: 27th January 2026

Revision History:

Name	Date	Description	Version
Team Wait Zero	27 Jan 2026	Initial SRS creation	1.0
Team Wait Zero	09 Feb 2026	Added Design, Security, and Traceability sections	1.1

Table of Contents

1.0 Introduction

1.1 Purpose

1.2 Scope

1.3 Glossary

1.4 References

1.5 Overview

2.0 Overall Description

2.1 System Environment

2.2 Functional Requirements Specification (Use Cases)

2.3 User Characteristics and classes

3.0 Requirements Specification

3.1 External Interface Requirements

3.2 Functional Requirements (Detailed Tables)

3.3 Non-functional Requirements

3.4 Software Quality Attributes

3.5 Other Requirements

4.0 Software Design

4.1 System Architecture

4.2 Detailed Design

4.2.1 Class Diagrams

4.2.2 Sequence Diagrams

4.3 Database Schema

4.4 Security Design

5.0 Traceability

5.1 Requirement-to-Design Mapping

1.0 Introduction

1.1 Purpose

The purpose of this document is to present a detailed description of the **Healthcare Management System with Live Token & Appointment Scheduling**. It explains the purpose, features, interfaces, and constraints of the system. It specifically details the **Offline-First** architecture designed to handle queue management in areas with unstable internet connectivity.

1.2 Scope

The system is a "Queue & Appointment Orchestration Platform" for clinics.

- **Goal:** Eliminate physical queues by allowing remote booking and live token tracking.
- **Key Feature:** The system functions in **Offline Mode** using a local database (SQLite) and synchronizes with the central server when connectivity is restored.
- **Users:** Patients, Doctors, and Receptionists.

1.3 Glossary

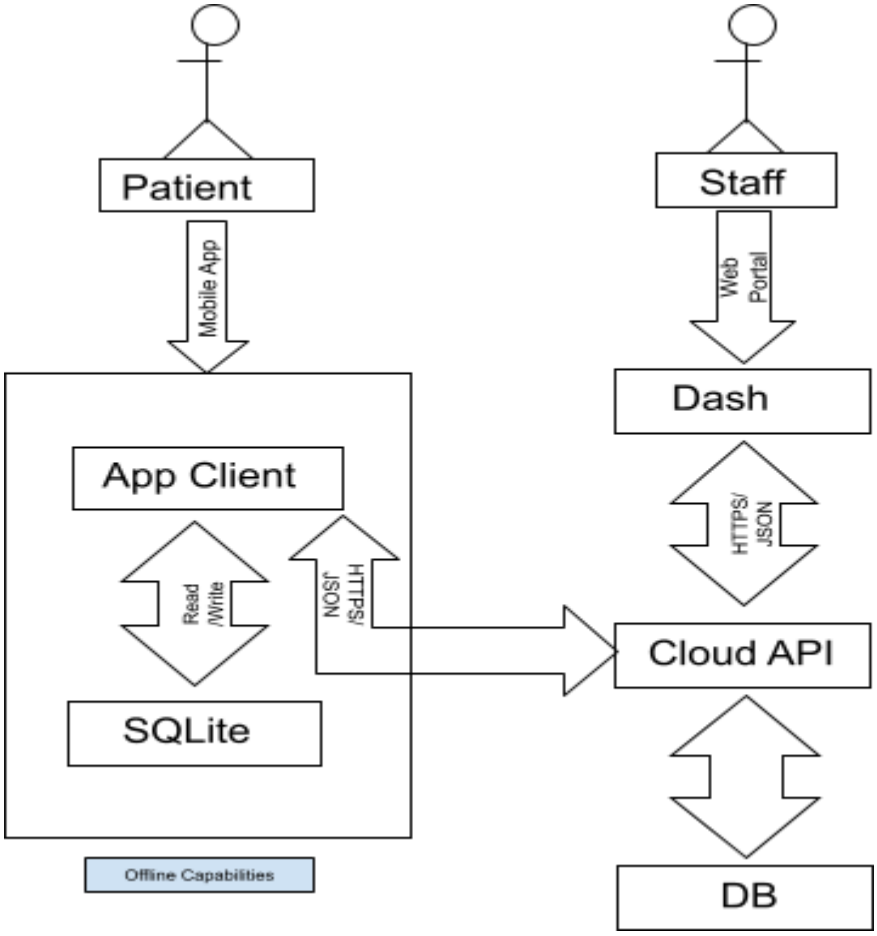
- **Appointment:** A scheduled consultation slot between a Patient and a Doctor
- **Token:** A digitally generated queue identifier assigned to a patient
- **Queue Orchestration:** Rule-driven management of patient flow using tokens and system constraints
- **Emergency Queue:** A separate, restricted queue for doctor-declared emergency cases
- **Check-in:**
  - Patient presence confirmation using GPS or manual validation
  - Can occur automatically after booking
- **Skip:** A situation where a patient is absent when their token is called
- **Auto-Cancellation:** System-driven cancellation after exceeding the allowed number of skips
- **Clinic Chain:** A group of clinics operating under a single owner
- **Audit Log:** Immutable record of system actions maintained for accountability
- **Role-Based Access Control (RBAC):** Permission system that restricts actions based on user roles

2.0 Overall Description

2.1 System Environment

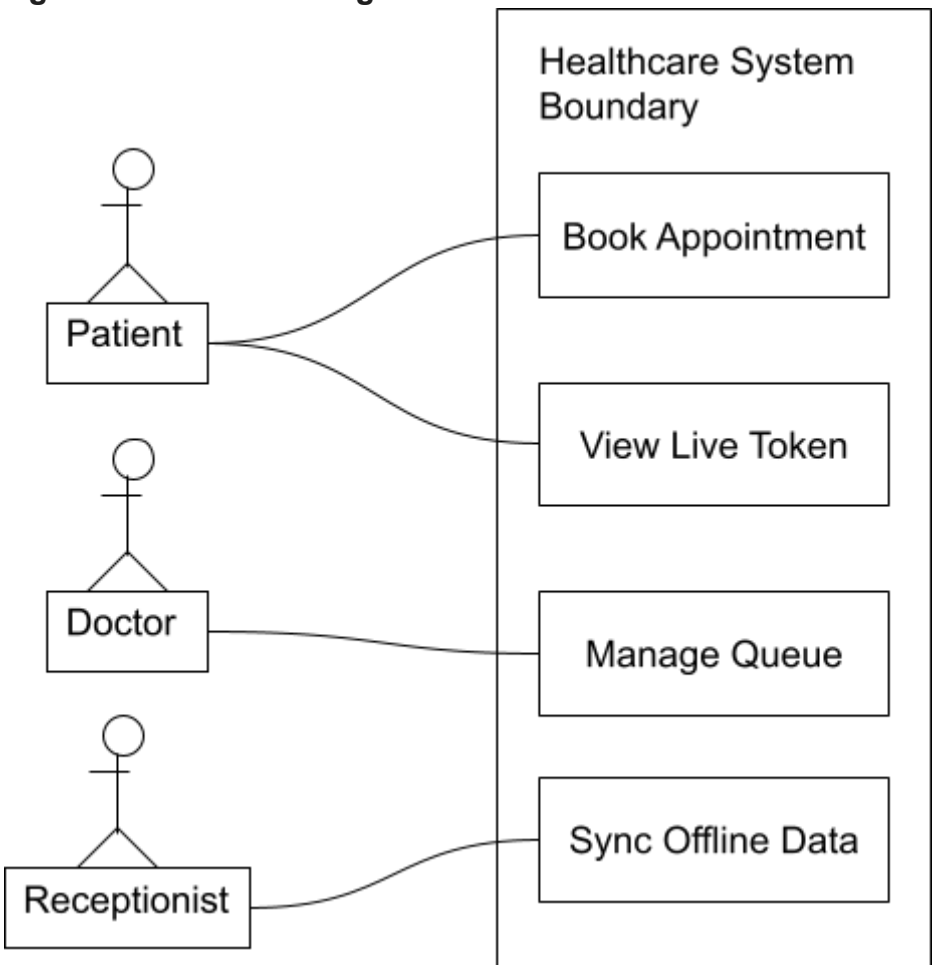
The system consists of a Mobile App (Patient), a Web Dashboard (Clinic Staff), and a Cloud Backend.

Figure 1 - System Environment



2.2 Functional Requirements Specification (Use Cases)

Figure 2 - Use Case Diagram



2.3 User Characteristics and classes

The system supports multiple user classes, each with distinct responsibilities and authority levels.

2.3.1 Patients

• Primary beneficiaries of the system
• Can book appointments with or without login
• Can join queues remotely and perform GPS-based check-in
• Can view live queue status, token position, and waiting time estimates
• Have access to their complete prescription history (read-only)

Patients are assumed to have basic digital literacy and access to a smartphone or internet- enabled device.

2.3.2. Doctors

• Medical professionals providing consultation
• Operate within strict system-imposed scheduling rules
• Cannot pause or delay queues
• Can manually call the next token
• Are the only authority permitted to declare emergencies
• Can create and edit prescriptions

Doctors are expected to follow system workflows to ensure fairness and operational discipline.

2.3.3. Receptionists

• Operational users managing clinic front-desk activities
• Assist patients with bookings and check-ins
• Monitor live queues and token status
• Cannot override queue rules or emergency limits
• Act as facilitators, not decision-makers

Receptionists are assumed to have basic system training.

2.3.4 Admin

• System-level authority
• Manages users, roles, clinics, and configuration parameters
• Resolves conflicts such as double-booking or system anomalies
• Oversees audit logs and compliance-related configurations

Admins do not interfere with live queue flow unless exceptional conditions arise.

2.3.5 Clinic Owner

• Oversees one or more clinics
• Accesses centralized dashboards for operational insights
• Cannot modify individual patient queues
• Focuses on performance, utilization, and system health metrics

3.0 Requirements Specification

3.1 External Interface Requirements

- **User Interfaces:** Responsive Mobile UI for patients; Web Dashboard for doctors.
- **Software Interfaces:** SQLite for local storage; REST API for server communication.

3.2 Functional Requirements

3.2.1 Book Appointment (Offline Capable) 3.2.2 Sync Data (The "How" of Offline)

Field	Description
Use Case ID	UC-01
Use Case Name	Book Appointment
Trigger	The patient selects a doctor and time slot.
Precondition	The user is logged in.
Basic Path	1. Patient clicks "Book". 2. The system sends requests to the server. 3. Server confirms slot availability. 4. Server generates Token ID. 5. The app displays "Booking Confirmed".
Alternative Paths	(Offline Mode) In basic path after Step 2, if No Internet: 2a. System saves appointment details to Local SQLite Database with status PENDING. 2b. The system generates a temporary local Token ID. 2c. System alerts user: "Saved Offline. Will sync automatically." 2d. Resume at basic path Step 5.
Postcondition	Appointment is saved (either locally or on server).

3.2.2 Sync Data (The "How" of Offline)

Field	**Description**
Use Case ID	UC-02
Use Case Name	Background Data Synchronization
Trigger	1. Internet connection is restored (Auto-detected). 2. User clicks “Sync Now” button.
Precondition	The local database has records with status PENDING or DIRTY.
Basic Path	1. The system detects the network. 2. System queries Local DB for PENDING records. 3. The system sends a batch POST request to Server API. 4. Server validates data and assigns permanent IDs. 5. Server returns success response. 6. System updates Local DB status to SYNCED.
Alternative Paths	(Conflict) If Server returns "Slot Already Taken": 4a. System marks record as CONFLICT. 4b. The app prompts user to select a new time slot.
Postcondition	Local and Server databases are consistent.

3.2.3 Call Next Token

Field	**Description**
Use Case ID	UC-03
Use Case Name	Call Next Token

Trigger	The doctor clicks "Next Patient".
Basic Path	1. The doctor clicks "Next". 2. System updates current token status to COMPLETED. 3. The system calculates the next token in the queue. 4. Live Queue updates for all patients.

3.2.4 Declare Emergency (Critical Feature)

Field	**Description**
Use Case ID	UC-04
Use Case Name	Declare Emergency
Trigger	The doctor presses the "Emergency" button on the dashboard.
Precondition	Doctor is logged in; Queue is active.
Basic Path	1. Doctor confirms emergency declaration. 2. The system freezes the current Standard Queue. 3. The system activates "Emergency Mode" on all displays. 4. System notifies waiting patients: "Queue Delayed due to Emergency". 5. The system logs the emergency start time.
Alternative Paths	(Quota Exceeded) If Doctor has exceeded daily emergency limit: 2a. System displays warning: "Daily Limit Reached. Admin Override Required." 2b. The doctor requests Admin approval. 2c. Admin approves. 2d. Resume at Step 2.

Postcondition	The regular queue is paused; an emergency case is being served.

3.2.5 GPS Check-In (Presence Validation)

Field	**Description**
Use Case ID	UC-05
Use Case Name	Patient Check-In
Trigger	The patient arrives at the clinic and opens the app.
Precondition	Appointment is scheduled for today; GPS is enabled.
Basic Path	1. Patient clicks "Check-In". 2. System retrieves device geolocation. 3. The system calculates distance to Clinic coordinates. 4. If distance < 100 meters, Check-in is successful. 5. Token status updates to "Arrived".
Alternative Paths	(GPS Failure/Spoofing) If distance > 100m or GPS fails: 4a. The system shows an error: "You are too far from the clinic." 4b. Patient approaches Receptionist. 4c. The receptionist manually overrides check-in. 4d. System logs "Manual Override" for audit.
Postcondition	Patient is marked as present in the queue.

3.2.6 Issue Digital Prescription

Field	**Description**
Use Case ID	UC-06

Use Case Name	Issue Prescription
Trigger	Consultation is completed.
Precondition	Patient token status is "In Consultation".
Basic Path	1. Doctor selects "Issue Prescription". 2. The system loads patient history. 3. The doctor enters diagnosis and medication. 4. The doctor digitally signs the form. 5. System saves prescription to Server (or Local DB if offline). 6. The patient receives a digital copy via App/SMS.
Alternative Paths	(Offline Mode) In Step 5, if Offline: 5a. System saves Prescription to Local SQLite (Encrypted). 5b. System marks record as DIRTY. 5c. Sync occurs automatically when the network restores.
Postcondition	Prescription is stored and accessible to the patient.

3.3 Non-Functional Requirements

This section defines how well the system must operate rather than what it does. For your project, non-functional requirements are not secondary — they are critical because the system is intended for real healthcare usage, startup deployment, and large-scale environments.

3.3.1 Performance Requirements

The system must perform reliably under high patient volume while maintaining real-time queue accuracy.

Requirements:

- **REQ-3.3.1.1:** The system shall support 5000+ concurrent users without
- degradation of core functionality.
- **REQ-3.3.1.2:** The system shall respond to user actions (booking, queue updates,

- check-in) within ≤ 3 seconds under normal load.
- **REQ-3.3.1.3:** Live queue updates shall propagate to all connected clients within ≤ 5 seconds.
- **REQ-3.3.1.4:** Token generation shall occur instantly upon appointment confirmation.
- **REQ-3.3.1.5:** Emergency queue operations shall not delay normal queue updates.

3.3.2 Safety Requirements

The system must ensure that no patient is disadvantaged or harmed due to system malfunction or misuse.

Requirements:

- **REQ-3.3.2.1:** The system shall prevent unauthorized access to medical and prescription data.
- **REQ-3.3.2.2:** The system shall ensure that emergency cases are never blocked by normal queue rules.
- **REQ-3.3.2.3:** Queue auto-correction mechanisms shall not skip or discard valid patients.
- **REQ-3.3.2.4:** Manual overrides shall be logged and restricted to authorized roles only.

3.3.3 Security Requirements

The system must meet real healthcare-grade security expectations.

Requirements:

- **REQ-3.3.3.1:** The system shall encrypt all sensitive data at rest.
- **REQ-3.3.3.2:** The system shall encrypt all data in transit using secure communication protocols.
- **REQ-3.3.3.3:** The system shall enforce strong authentication mechanisms (password, OTP).
- **REQ-3.3.3.4:** The system shall enforce strict role-based authorization.
- **REQ-3.3.3.5:** The system shall maintain audit logs for:
    - Login attempts
    - Prescription edits
    - Emergency declarations
    - Admin interventions
- **REQ-3.3.3.6:** Audit logs shall be tamper-resistant and retained as per configuration.

3.4 Software Quality Attributes

This subsection defines the expected quality characteristics of the system.

3.4.1. Reliability

• The system shall maintain ≥ 99% availability during clinic operating hours.

• The system shall recover queue state accurately after restarts.

3.4.2. Scalability

• The system shall support horizontal scaling without redesign.

• Adding new clinics shall not affect existing clinic operations.

3.4.3. Maintainability

• Business rules (skip limits, emergency quotas) shall be configurable without code changes.

• Modular design shall allow independent feature upgrades.

3.4.4. Usability

• Interfaces shall be intuitive for non-technical users.

• Critical actions shall require confirmation to prevent mistakes.

3.4.5. Portability

• The system shall be deployable on cloud or on-premise environments.

• Client interfaces shall function across devices and screen sizes.

3.5 Other Requirements

3.5.1. Audit, Monitoring, and Logging Requirements

To ensure accountability, transparency, and traceability, the system shall maintain detailed logs of critical actions.

Requirements:

- **REQ-3.5.1.1:** The system shall log all user authentication events.
- **REQ-3.5.1.2:** The system shall log all appointment creation, cancellation, and auto- cancellation events.
- **REQ-3.5.1.3:** The system shall log all token movements including:
    - Push-to-end actions
    - Auto-cancellations after skip limit
    - Emergency queue insertions

- **REQ-3.5.1.4:** The system shall log all prescription creation and modification actions.
- **REQ-3.5.1.5:** Each log entry shall include timestamp, user role, user ID, and action performed.
- **REQ-3.5.1.6:** Logs shall be accessible only to Admins and authorized Clinic Owners.

3.5.2. Configuration and Customization Requirements

The system must support configuration without redeployment, enabling adaptation across different clinics and regions.

Configurable Parameters:

- Maximum allowed skips before auto-cancellation
- Emergency quota per doctor or clinic per day
- GPS check-in radius
- Check-in time window
- Appointment slot duration
- Notification thresholds

Requirements:

- **REQ-3.5.2.1:** The system shall allow Admins to modify configuration parameters through the Admin Interface.
- **REQ-3.5.2.2:** Configuration changes shall take effect without system downtime.
- **REQ-3.5.2.3:** Configuration changes shall not affect already active queues.

3.5.3. Legal, Ethical, and Compliance Considerations

While the system remains generic and region-agnostic, it is designed to respect healthcare data ethics.

Requirements:

- **REQ-3.5.3.1:** The system shall restrict access to patient data strictly based on role.
- **REQ-3.5.3.2:** The system shall ensure patient consent for data usage where applicable.
- **REQ-3.5.3.3:** Prescription data shall not be shared across clinics unless explicitly permitted.
- **REQ-3.5.3.4:** Data retention policies shall be configurable.

3.5.4. Deployment Requirements

The system is intended for flexible deployment environments.

Requirements:

- **REQ-3.5.4.1:** The system shall support cloud-based deployment.
- **REQ-3.5.4.2:** The system shall support on-premise deployment for clinics with regulatory constraints.
- **REQ-3.5.4.3:** Deployment shall support horizontal scaling.

- **REQ-3.5.4.4:** System updates shall not disrupt active queues.

3.5.5. Backup and Data Recovery Requirements
Reliable backup and recovery mechanisms are essential for healthcare systems.

Requirements:
- **REQ-3.5.5.1:** The system shall perform automated periodic backups.
- **REQ-3.5.5.2:** Backups shall include appointment, token, queue, and prescription data.
- **REQ-3.5.5.3:** Backup restoration shall preserve queue fairness rules.
- **REQ-3.5.5.4:** Restoration actions shall be logged and auditable.

3.5.6. Future Enhancements and Scalability Considerations
The system is designed with future growth in mind.

Planned Enhancements:
- Native mobile applications (Android and iOS)
- Advanced analytics dashboards for clinic owners
- Integration with government healthcare systems
- Inter-clinic appointment referrals
- AI-assisted (rule-validated) wait-time insights

Important Constraint:
Future enhancements shall never compromise the rule-driven fairness core of the system.

4.0 Software Design

4.1 System Architecture

The system follows a **Hybrid Client-Server Architecture**.

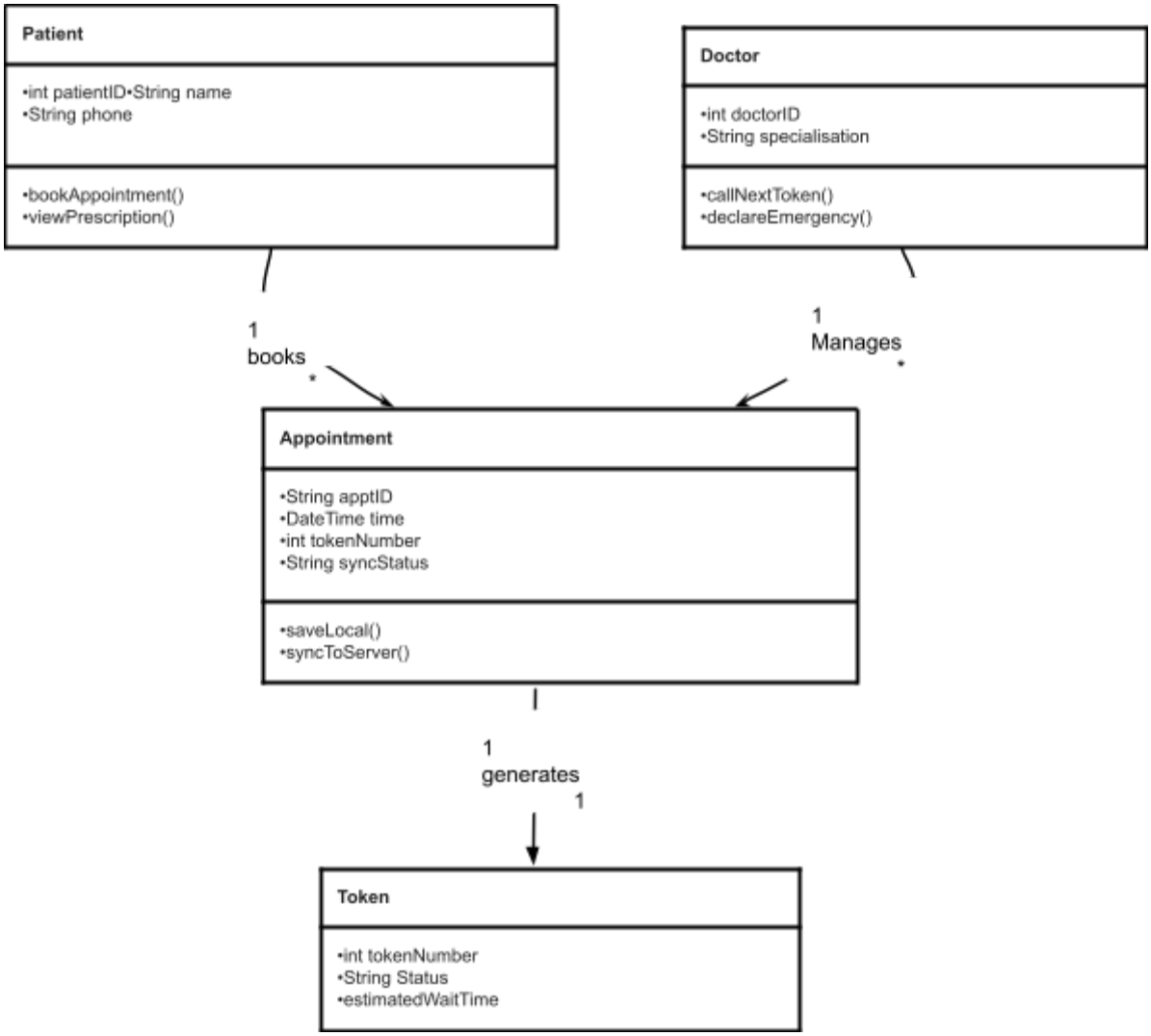
- **Client Layer:** Flutter Mobile App (Android/iOS) with local SQLite storage.
- **API Layer:** Node.js Express Server handling REST requests.
- **Data Layer:** MySQL (Central) and SQLite (Local).
- **Sync Engine:** A dedicated service module responsible for bi-directional data replication.

4.2 Detailed Design

4.2.1 Class Diagrams

The following diagram illustrates the core object structure and relationships.

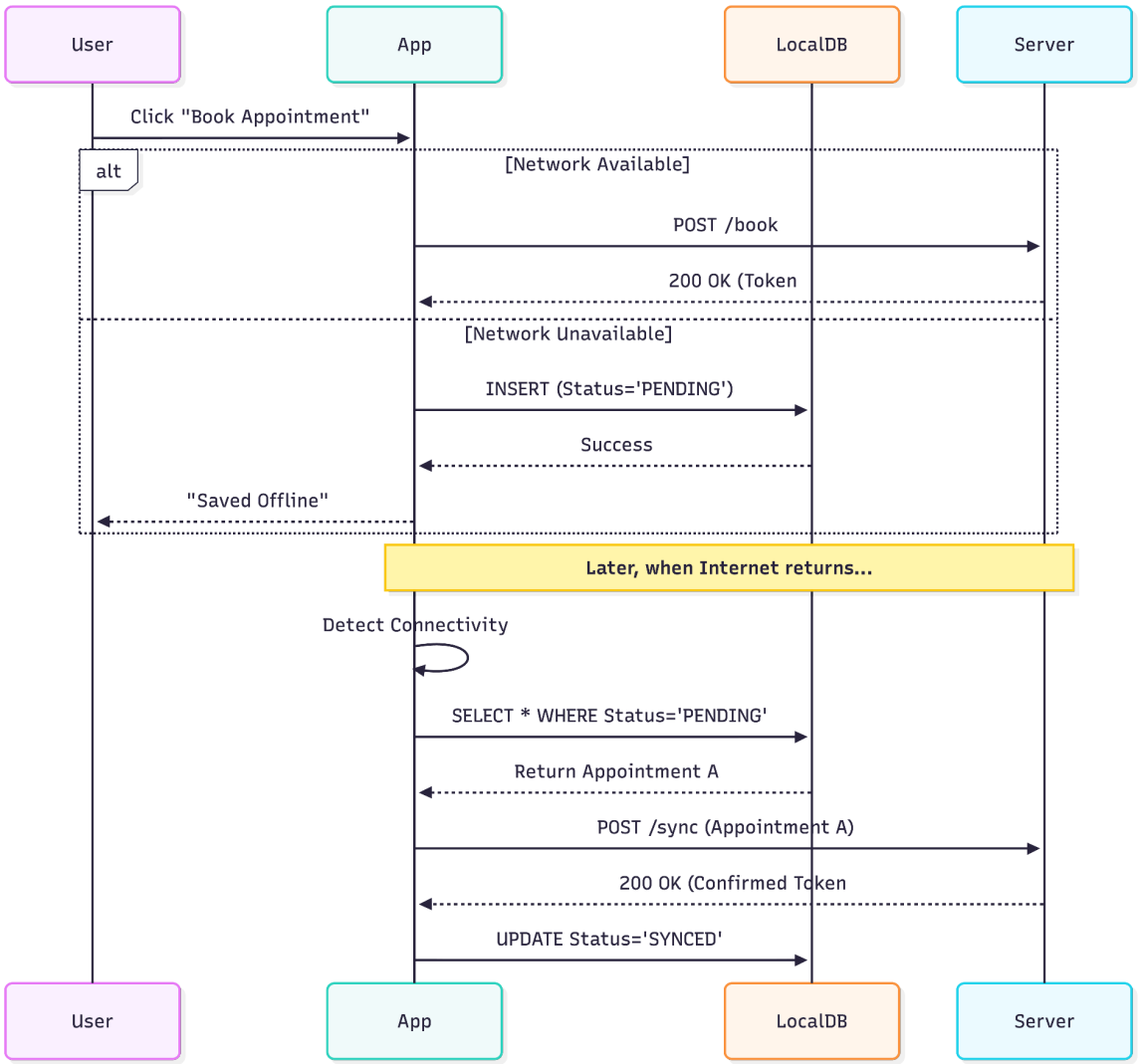
Figure 3 - Class Diagram



4.2.2 Sequence Diagrams

This diagram details the specific flow for **Offline Booking and Synchronization**, answering "how it is done."

Figure 4 - Offline Sync Sequence Diagram



4.3 Database Schema

Table: Appointments (Local SQLite & Server MySQL)

Column Name	Data Type	Constraints	Description
appt_id	UUID	PK	Unique ID (generated locally to avoid collision).
patient_id	INT	FK	Link to Patient.

doctor_id	INT	FK	Link to Doctor.
token_no	INT		Assigned by server (Null if offline).
sync_status	ENUM		Values: Synced, Pending, Conflict.
created_at	TIMESTAMP		Used for "Last Write Wins" resolution.

4.4 Security Design

4.4.1 Authentication and Authorization Mechanisms

- **JWT (JSON Web Tokens):** Used for stateless session management.
- **Offline Auth:** An encrypted session token is stored in the device's secure keystore, allowing the user to open the app and view data even without the internet.

4.4.2 Security Protocols

- **SSL/TLS:** All data in transit (Sync process) is encrypted via HTTPS.
- **SQLCipher:** The local SQLite database is encrypted to prevent data extraction if the phone is stolen.

5.0 Traceability

5.1 Requirement-to-Design Mapping

The following table maps the Functional Requirements (Section 3) to the specific Design Artifacts (Section 4) to ensure all requirements are addressed.

Requirement / Use Case ID	**Requirement / Use Case Name**	**Design ID(s)**	**Design Artifact**
UC-01	Book Appointment (Offline)	D-CL-01, D-DB-01, D-SQ-01	Class Diagram (Patient), DB Schema (Appointments), Offline Sync Sequence
UC-02	Background Data Sync	D-ARC-01, D-SQ-01	System Architecture (Sync Engine), Offline Sync Sequence

UC-03	Call Next Token	D-CL-02, D-DB-01	Class Diagram (Doctor), DB Schema (Token Logic)
UC-04	Declare Emergency	D-CL-02, D-SQ-02	Class Diagram (Doctor), Emergency Flow Sequence
UC-05	GPS Patient Check-In	D-CL-01, D-ARC-02	Class Diagram (Patient), Location Service Module
UC-06	Issue Prescription	D-DB-02, D-SEC-02	DB Schema (Prescriptions), Security (Digital Signature)
NFR-01	Data Security (Encryption)	D-SEC-01	Security Protocols (SQLCipher/SSL)
NFR-02	Performance (<2 sec load)	D-ARC-03	System Architecture (Local Caching Strategy)